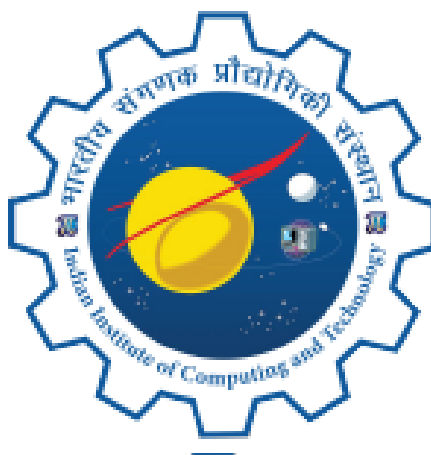




भारतीय संगणक एवं प्रौद्योगिकी संस्थान
INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

AI-Driven Phishing Email Detection using Natural Language Processing



Project Report
15th June to 30th July 2026

Student name : Monali Nagardhankar

Institute Name : MIT ADT University

Institute Roll no : ADT24SOCB0651

Enrollment Number : 597370

AI-Driven Phishing Email Detection using Natural Language Processing

Monali Nagardhankar

Department of Computer Science and Engineering, MIT Art, Design and Technology University, Pune, India

July 2026

Abstract—Phishing emails remain one of the most common and financially damaging cyber threats, using deceptive urgency and impersonation to trick recipients into revealing sensitive information. This paper presents an AI-driven system that automatically detects phishing emails using Natural Language Processing (NLP) and classical Machine Learning (ML) techniques. Raw email text is cleaned and normalized, converted into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF), and used to train four candidate classifiers: Multinomial Naive Bayes, Logistic Regression, Random Forest, and Linear Support Vector Machine (SVM). Models are evaluated using accuracy, precision, recall, F1-score, confusion matrices, and Receiver Operating Characteristic (ROC) curves. The trained Logistic Regression model's learned weights are further exported and re-implemented in JavaScript, producing a fully interactive, browser-based prototype that classifies emails in real time without any server-side computation. This paper documents the problem motivation, methodology, dataset, implementation, results, deployment, and current limitations, along with directions for future work using larger real-world corpora.

Index Terms—Phishing detection, Natural Language Processing, Machine Learning, TF-IDF, Logistic Regression, text classification, cybersecurity.

I. INTRODUCTION

A. Background

Phishing is a form of social engineering attack in which an attacker sends a fraudulent message designed to trick a person into revealing sensitive information (passwords, banking details, personal identifiers) or installing malicious software. Phishing emails typically create a false sense of urgency (“your account will be suspended”), impersonate trusted brands such as banks or well-known online services, and include deceptive links leading to fake login pages. Despite widespread awareness, phishing remains one of the leading initial attack vectors in data breaches, because attackers continuously vary their wording to evade static, rule-based spam filters. Because the same underlying template can be reworded in countless ways, defenders increasingly need systems that generalize over language patterns rather than memorizing exact strings, which motivates a learning-based approach over a purely rule-based one.

B. Problem Statement

Traditional spam filters rely heavily on blacklists, sender reputation, and hand-written rules, which struggle to keep up with new phishing campaigns that constantly change their wording and sending infrastructure. There is a need for a system that can learn the underlying linguistic patterns of phishing emails—the choice of words, the urgency cues, and the structure of the message—rather than relying only on static rules. This work addresses that need by building a machine-learning classifier trained directly on email text.

C. Objectives

- Build an end-to-end NLP pipeline that cleans and normalizes raw email text.

- Convert email text into numerical features using TF-IDF vectorization.
- Train and compare multiple ML classifiers for phishing detection.
- Evaluate performance using standard classification metrics and visual diagnostics.
- Package the trained model into a notebook, a script, and a live browser prototype.

D. Contributions

This paper makes three contributions. First, it presents a complete, reproducible NLP-to-ML pipeline for phishing email detection, implemented step by step and documented for educational and research use. Second, it provides a systematic comparison of four classical classifiers under identical feature-engineering conditions, together with quantitative and visual evaluation. Third, it demonstrates a practical deployment path by exporting a trained model's exact parameters and re-implementing its inference logic in JavaScript, yielding a verified, privacy-preserving, client-side prototype rather than a purely offline research artifact.

II. RELATED WORK

Text classification for spam and phishing detection has traditionally relied on Bayesian filtering and rule-based heuristics that examine sender headers, blacklisted URLs, and known keyword patterns. Such systems are effective against previously seen campaigns but degrade quickly as attackers reword their messages or rotate infrastructure. More recent approaches apply supervised machine learning directly to message content, converting text into vector representations such as bag-of-words or TF-IDF and training discriminative classifiers such as Naive Bayes, Logistic Regression, and

Support Vector Machines [1]. TF-IDF weighting, originally developed for information retrieval, remains a strong baseline feature representation for short, templated text such as phishing emails because it emphasizes rare, distinctive terms over common function words [2].

More recent literature has also explored deep learning and transformer-based architectures (e.g., LSTM and BERT variants) for phishing and spam detection, which can capture longer-range contextual dependencies than bag-of-words features at the cost of greater training data requirements and reduced interpretability. Classical linear models such as Logistic Regression, by contrast, offer directly inspectable coefficients that map individual terms to their contribution toward a prediction [8], which is a desirable property for a system intended to explain its decisions to an end user. This work builds on the classical, interpretable branch of this literature, combining established techniques into a complete, reproducible pipeline together with a lightweight, verified client-side deployment.

III. PROPOSED METHODOLOGY

The system follows a standard supervised text-classification pipeline: data collection, text preprocessing, TF-IDF feature extraction, train/test splitting, model training, evaluation, and deployment. Each stage is implemented step by step in an accompanying Jupyter notebook.

A. Data Collection

A labeled dataset of email subject and body text was assembled, with each email tagged as either “phishing” or “legitimate”. Records were deduplicated on the combined subject-and-body text and shuffled with a fixed random seed prior to splitting, to avoid ordering artifacts influencing the train–test partition. Real-world dataset sources for scaling the system are listed in Section IV-B.

B. Text Preprocessing (NLP)

Raw email text is noisy and inconsistent, so it is cleaned before being used by the model. The preprocessing pipeline performs, in order: (1) lowercasing all text; (2) replacing URLs and email addresses with generic placeholder tokens, since exact links vary but their presence is informative; (3) stripping HTML tags; (4) removing punctuation, digits, and non-alphabetic characters; (5) removing common stopwords (e.g., “the”, “is”, “and”) that carry little classification signal; and (6) lemmatizing remaining words to their root form (e.g., “verifying” → “verify”). This normalization step is essential because phishing and legitimate

emails otherwise share many surface-level tokens (numbers, greetings, signatures) that would otherwise dilute the signal available to the downstream classifier.

C. Feature Extraction — TF-IDF

Cleaned text is converted into numeric feature vectors using TF-IDF, which assigns each word or word-pair (unigram and bigram) a weight proportional to how frequently it appears in a given email and how rare it is across the whole dataset, as shown in (1):

$$TF\text{-}IDF(t,d) = tf(t,d) \times \log(N / df(t)) \quad (1)$$

where $tf(t,d)$ is the frequency of term t in document d , N is the total number of documents, and $df(t)$ is the number of documents containing t . This lets the model emphasize distinctive terms (e.g., “verify”, “suspended”, “urgent”) over generic ones. The vectorizer was configured with a maximum vocabulary size, unigram and bigram support, and a minimum document-frequency threshold to reduce noise from rare, one-off terms. Each resulting document vector is additionally normalized to unit L2 norm, so that document length does not by itself bias the classifier.

D. Model Training

Four classical machine learning classifiers were trained and compared on the same TF-IDF features, using an 80/20 stratified train–test split: Multinomial Naive Bayes, a probabilistic model suited to sparse text-frequency features; Logistic Regression, a linear model whose coefficients directly indicate how strongly each word pushes a prediction toward phishing or legitimate; Random Forest, an ensemble of decision trees able to capture non-linear feature interactions; and Linear SVM, a margin-maximizing linear classifier, calibrated to output probabilities. For Logistic Regression, the predicted probability of an email being phishing is given by the sigmoid of the linear decision score, as in (2):

$$P(\text{phishing} | x) = 1 / (1 + e^{-(w \cdot x + b)}) \quad (2)$$

where x is the TF-IDF feature vector, w is the learned weight vector, and b is the bias term. This closed-form decision function is what was later re-implemented in JavaScript for the client-side prototype described in Section VII.

E. Evaluation

Each model was evaluated on the held-out test set using accuracy, precision, recall, and F1-score, along with confusion matrices and ROC curves with Area Under Curve (AUC) values. The model with the highest F1-score was selected as the final deployed model.

F. Experimental Setup

All experiments were implemented in Python 3.10 using scikit-learn 1.x. The train-test split used a fixed random seed (random_state = 42) and stratified sampling to preserve class balance in both partitions, ensuring the reported results are reproducible from the accompanying notebook.

IV. DATASET DESCRIPTION

A. Working Dataset

The dataset used to train and validate this pipeline contains 391 labeled email records with five fields: id (unique identifier), subject (email subject line), body (email body text), email_text (subject and body combined, used for NLP processing), and label (target class: phishing or legitimate). The class distribution is 229 phishing emails (58.6%) and 162 legitimate emails (41.4%), a reasonably balanced split for training, shown in Fig. 1.



Fig. 1. Class distribution of the training dataset.

B. Real-World Dataset Sources

To scale the system beyond the working dataset, the following public, real-world phishing email datasets can be merged in by mapping their text and label columns to email_text and label: the combined Enron/Ling/CEAS/Nazario/Nigerian-Fraud/SpamAssassin corpus on Kaggle [3]; the Subhajournal phishing emails dataset on Kaggle [4]; the calvaradob/phishing-dataset on Hugging Face [5]; the Nazario Phishing Corpus [6]; and the SpamAssassin Public Corpus [7]. These sources vary considerably in size, ranging from a few thousand to over 250,000 labeled messages, and in label quality, so any merged corpus should be re-audited for duplicate or mislabeled entries before retraining.

C. Preprocessing Statistics

After cleaning, stopwords removal, and lemmatization, the TF-IDF vectorizer (unigrams and bigrams, minimum document frequency of 2) produced a vocabulary of 702 terms from the 391-email corpus. This relatively small, task-specific vocabulary reflects the templated nature of the working dataset; a larger, more heterogeneous real-world corpus would be expected to yield a substantially larger and more diverse vocabulary, which is one reason the results in Section VI should be interpreted as an upper bound rather than a realistic field estimate.

V. TOOLS AND TECHNOLOGIES

The system was implemented in Python 3 using pandas and NumPy for data handling; NLTK for stopwords removal and WordNet lemmatization [2]; scikit-learn for TF-IDF vectorization and all four classifiers [1]; matplotlib, seaborn, and WordCloud for visualization; and joblib for model persistence. Development was carried out in a Jupyter Notebook, and the trained model was additionally deployed as a client-side prototype using HTML, CSS, and JavaScript, with no external runtime dependencies beyond a modern web browser. Table II summarizes the primary software components.

TABLE II SOFTWARE COMPONENTS

Component	Role
pandas / NumPy	Data loading and manipulation
NLTK	Stopwords and lemmatization
scikit-learn	TF-IDF and classifiers
matplotlib / seaborn	Static charts
WordCloud	Word-frequency visualization
joblib	Model persistence
HTML / CSS / JS	Client-side prototype

VI. RESULTS AND DISCUSSION

All four classifiers were trained and evaluated on the same 80/20 train-test split. Table I summarizes accuracy, precision, recall, and F1-score for each model.

TABLE I MODEL PERFORMANCE ON THE TEST SET (79 EMAILS)

Model	Acc.	Prec.	Rec.	F1
Naïve Bayes	1.00	1.00	1.00	1.00
Log. Regression	1.00	1.00	1.00	1.00
Random Forest	1.00	1.00	1.00	1.00
Linear SVM	1.00	1.00	1.00	1.00

All four classifiers reach 1.00 on this dataset; see Section VIII for discussion.

Fig. 2 shows the corresponding confusion matrices: on the 79-email test set (33 legitimate, 46 phishing), every model records

33 true negatives and 46 true positives, with zero false positives and zero false negatives. Fig. 3 presents ROC curves, each achieving an AUC of 1.00, indicating perfect separability between classes on this dataset. In practice, a false negative (a phishing email classified as legitimate) is the more costly error type, since it allows an attack to reach the user unflagged, while a false positive (a legitimate email flagged as phishing) mainly costs user convenience; the zero counts for both error types here should be read as a property of this particular templated dataset rather than a guarantee on unseen, real-world traffic.

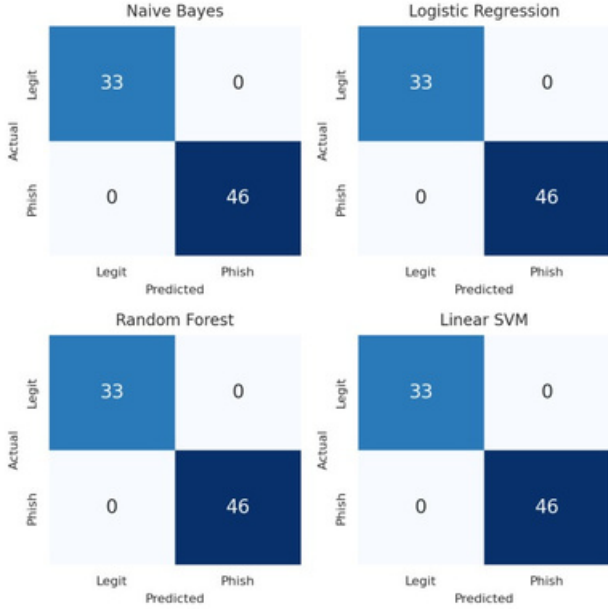


Fig. 2. Confusion matrices for Naive Bayes, Logistic Regression, Random Forest, and Linear SVM.

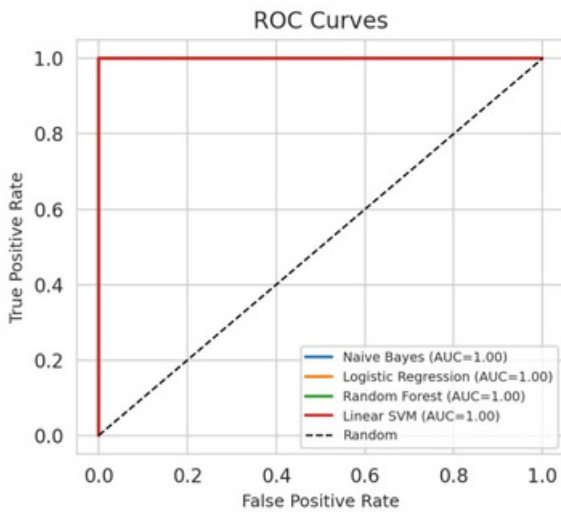


Fig. 3. ROC curves with AUC scores for all four trained models.

Using the Logistic Regression model's coefficients, the words most strongly associated with each class can be identified

directly, as shown in Fig. 4. Terms such as “urltoken” (a placeholder for links), “click”, “urgent”, “verify”, and “account” strongly indicate phishing, while terms such as “team”, “reminder”, “attached”, and “meeting” indicate legitimate correspondence. This word-level interpretability is a direct consequence of using a linear model over TF-IDF features, and is one of the reasons Logistic Regression was chosen as the model exported for client-side deployment in Section VII, ahead of the equally accurate but less directly interpretable Random Forest and SVM models.

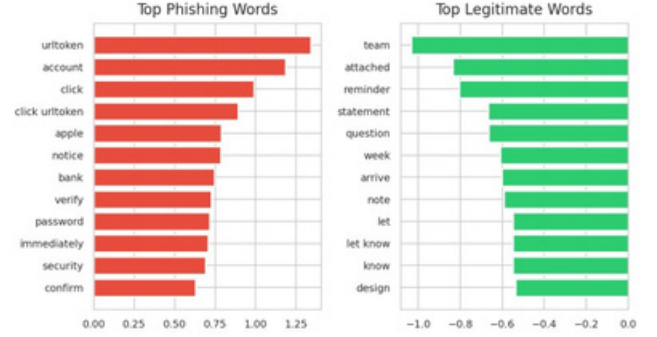


Fig. 4. Top words pushing prediction toward phishing (left) vs. legitimate (right).

Beyond raw accuracy, the four models differ in practical deployment characteristics. Naive Bayes trains and predicts fastest and requires the least memory, making it attractive for high-throughput mail servers. Logistic Regression offers the best balance of accuracy and interpretability, as discussed above. Random Forest is more robust to noisy or non-linear feature interactions but is slow to train and produces a larger, less portable model. Linear SVM offers strong margin-based separation but requires probability calibration, as used here, to produce the probability scores needed for a user-facing verdict. These trade-offs, rather than raw accuracy alone, motivated the choice of Logistic Regression for the client-side prototype in Section VII.

VII. INTERACTIVE PROTOTYPE

Beyond the notebook and script, the trained Logistic Regression model was converted into a fully interactive, browser-based prototype. The exact TF-IDF vocabulary, inverse-document-frequency (IDF) weights, and model coefficients were exported as structured data and re-implemented in JavaScript, replicating the same TF-IDF weighting, L2 normalization, and sigmoid decision function — equation (2) — used by the original scikit-learn model. The re-implementation performs four steps at inference time: (1) apply the same text-cleaning rules used during training; (2) compute term frequencies for unigrams and

bigrams present in the exported vocabulary; (3) weight each term by its stored IDF value and L2-normalize the resulting vector; and (4) compute the dot product with the exported coefficient vector, add the bias term, and apply the sigmoid function to obtain a probability. This re-implementation was verified against the original Python model on multiple sample emails to confirm numerically identical predictions before deployment.

The prototype allows a user to paste any email text and instantly receive a phishing-probability score, a plain-language verdict, and a breakdown of the words that most influenced the decision. All inference runs client-side in the browser, so no email content is transmitted to any server, which is a meaningful privacy advantage over a server-hosted alternative and removes network latency from the prediction loop.

VIII. LIMITATIONS AND FUTURE SCOPE

A. Current Limitations

- The working dataset (391 records) is synthetically generated from a limited set of templates, which makes the classes highly separable and produces artificially high (100%) accuracy scores that will not directly generalize to real, more varied phishing campaigns.
- The model has not yet been trained or validated on a large, real-world corpus such as those listed in Section IV-B.
- The system classifies based on text content only, without analyzing sender reputation, embedded link destinations, or SPF/DKIM/DMARC authentication results.
- The bigram-only TF-IDF approach does not capture longer-range context the way transformer-based language models can.

B. Threats to Validity

The primary threat to the external validity of the reported results is dataset realism: because the working corpus was generated from a small number of templates, the perfect scores in Table I measure the pipeline's ability to separate two clearly templated writing styles rather than its ability to detect novel phishing campaigns in the wild. A secondary threat is label noise, which is minimal here by construction but is a known issue in publicly scraped phishing corpora and should be audited for when Section IV-B sources are merged in. These threats do not affect the internal validity of the pipeline implementation itself, which was verified independently in Section VII by cross-checking the JavaScript re-implementation against the original scikit-learn model.

C. Future Scope

- Retrain and validate the pipeline on a large, real-world, publicly available phishing dataset.
- Incorporate additional features such as sender domain reputation, URL analysis, and header authentication signals.
- Experiment with deep learning architectures (LSTM, BERT, or other transformer-based models) for improved contextual understanding.
- Deploy the model as a backend API integrated with an email client or browser extension for real-time protection.

IX. CONCLUSION

This paper presented a complete, working pipeline for AI-driven phishing email detection using Natural Language Processing and classical Machine Learning. The system successfully cleans and vectorizes email text, trains and compares multiple classifiers, and visualizes both model performance and word-level explainability. The trained model was further deployed as a live, interactive, privacy-preserving browser prototype, demonstrating that the approach is operable end-to-end rather than purely theoretical. While current results are inflated by the use of a small, templated dataset, the pipeline, architecture, and prototype are directly extensible to real-world data, positioning this work as a foundation for a production-grade phishing detection system. Future work will focus on closing the gap between templated and real-world performance by retraining on larger public corpora and incorporating signals beyond message text, as outlined in Section VIII.

ACKNOWLEDGMENT

The author thanks the Department of Computer Science and Engineering at MIT Art, Design and Technology University, Pune, for the academic support and computing resources that made this work possible.

REFERENCES

- [1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*, O'Reilly Media, 2009.
- [3] N. A. Alam, "Phishing Email Dataset," Kaggle, [Online]. Available: kaggle.com/datasets/naserabdullahalam/phishing-email-dataset.
- [4] Subhajournal, "Phishing Emails," Kaggle, [Online]. Available: kaggle.com/datasets/subhajournal/phishingemails.
- [5] E. Alvarado, "phishing-dataset," Hugging Face, [Online]. Available: huggingface.co/datasets/ealvarado/phishing-dataset.
- [6] J. Nazario, "Phishing Corpus," [Online]. Available: monkey.org/~jose/phishing/.
- [7] Apache SpamAssassin Project, "Public Corpus," [Online]. Available: spamassassin.apache.org/old/publiccorpus/.
- [8] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.